**ALGORITHM 3:** Source-DPOR algorithm considering locks.

**Initial call:** $Explore(\langle\rangle, \emptyset)$

```
1  Explore(E, Sleep)
2      foreach lock l do
3          foreach p whose next operation after E is lock(l) do
4              let e be the last event in E with op_[E](e) = lock(l);
5              let E' = pre(E, e);
6              let v = notdep(e, E).p;
7              if I_[E'](v) ∩ backtrack(E') = ∅ then
8                  add some q' ∈ I_[E'](v) to backtrack(E');
9      if ∃r ∈ (enabled(s_[E]) \ Sleep) then
10         backtrack(E) := {r};
11         while ∃p ∈ (backtrack(E) \ Sleep) do
12             foreach e ∈ dom(E) such that(e ≲_E.p next_[E](p)) do
13                 let E' = pre(E, e);
14                 let v = notdep(e, E).p;
15                 if I_[E'](v) ∩ backtrack(E') = ∅ then
16                     add some q' ∈ I_[E'](v) to backtrack(E');
17             let Sleep' = {q ∈ Sleep | E ⊢ p◇q};
18             Explore(E.p, Sleep');
19             add p to Sleep;
```

case: at line 6, the sequence $v$ is constructed as a sequence of events that can follow $E'$ in an execution where $p$ takes the lock before $\widehat{e}$. At line 7, it is tested whether the backtrack set at $E'$ already contains a process that makes such an extension of $E'$ possible, otherwise such a process is added at line 8.

*8.1.2 Correctness of Algorithm 3.* Let us next consider how to extend the proof of correctness for Algorithm 1 to become a proof of correctness for Algorithm 3. Here we present the needed modifications to the proof of Theorem 5.2. We thus consider how to extend the proof of Lemma 5.3.

The proof remains the same as in Section 5.2 up to Property (9), stated in the proof of Claim 5.4. This property, i.e., $E \vdash q.w_R$, is no longer true if there is a lock $l$ such that $next_{[E]}(q)$ and $e_q$ perform the operation `lock(l)`, since $\widehat{e_q}$ is now disabled after $E.q.w_q$. We therefore here explain how the proof should be continued for this case; all other cases can be handled as in Section 5.2.

We first infer, from $E \vdash q \diamond w_q$, that $w_q$ does not contain any operation on the lock $l$. We then replace Claim 5.5 by the following stronger one.

*For all $p \in done(E)$ with $p \neq q$ we have $p \notin WI_{[E]}(q.w_q)$.*

This property is actually shown in Cases 1 and 2a in the proof for Claim 5.5. The other cases, i.e., Cases 2b and 2c, are not applicable when $next_{[E]}(q)$ and $e_q$ both perform the operation `lock(l)`, since these cases assume $e_p = e_q$ and $E \vdash p \diamond q$, which is not possible when $e_q$ performs the operation `lock(l)`. This follows by noting that since the next events of $p$ and $q$ conflict with $e_q$, they must both be lock or unlock operations on $l$. Since $next_{[E]}(q)$ performs the operation `lock(l)`, we infer $E \nvdash p \diamond q$.